# Extended Abstract

**Motivation** Direct Preference Optimization (DPO) efficiently aligns language models but may suffer from overfitting when preference data covers only narrow distribution slices. DPO's KL regularization applies solely to preference examples, offering limited protection against distribution shift on broader tasks.

**Method** We extend DPO with KL regularization on non-preference data through two approaches: (1) Offline KL-DPO adds regularization using fixed non-preference examples: $\mathcal{L}$KL-DPO $=$ $\mathcal{L}$DPO $+ \lambda \mathbb{E} z \sim \mathcal{D}$general$[\text{KL}(\pi_\theta(z)|\pi_{\text{ref}}(z))]$. (2) Online KL-DPO dynamically samples rollouts during training: $\mathcal{L}$Online-KL-DPO $=$ $\mathcal{L}$DPO $+ \lambda \mathbb{E} y \sim \pi\theta[\text{KL}(\pi_\theta(y)|\pi_{\text{ref}}(y))]$, preventing hidden distribution-shift failures.

**Implementation** We fine-tuned Qwen2.5-0.5B on SmolTalk (460k examples) then applied DPO variants on 10% of UltraFeedback (6k examples) to simulate extreme data scarcity. All methods used $\beta = 0.1$, KL weight=1.0, with online sampling using temperature=1.0, $top_p$=0.8. Models were evaluated on win rate against SFT baseline and perplexity across multiple datasets.

**Results** Offline KL-DPO achieved superior performance with 65% win rate vs. SFT (compared to 62% for standard DPO) while maintaining lower perplexity on UltraFeedback (4.85 vs. 5.06). Offline KL-DPO showed intermediate results. Online regularization demonstrated more stable training dynamics.

**Discussion** Online KL regularization successfully balances preference alignment with distribution coverage, though at increased computational cost from rollout generation. The method's effectiveness in extreme data scarcity (10% of preference data) suggests promise for real-world scenarios with limited human feedback.

**Conclusion** We demonstrate that extending KL regularization beyond preference data significantly improves DPO robustness. Online sampling during training provides the best trade-off between alignment quality and generalization, offering a practical solution for preference learning under data constraints.

# Online KL-regularization for DPO

**Sophie Ostmeier**
Department of Computer Science
Stanford University
sostm@stanford.edu

**V. Nathan Josephs**
Department of Computer Science
Stanford University
vnjosephs@stanford.edu

## Abstract

We often observe that DPO training can cause a model to overfit and reduce performance on datasets that differ substantially from the preference dataset. One hypothesis for the cause of this is that most preference data only covers parts of the original distribution even though Direct Preference Optimization (DPO) assumes that the preference data comes from the *entire* distribution of interest. This is usually not true, as the preference labels focus usually on a specific, easy to label, slice sometimes filtering examples where we cannot tell the difference. We extend the implicit KL regularization in the DPO objective to the entire distribution of interest. We explore offline and online KL regularization through sampling examples or rollouts during training from the entire distribution of interest. We hypothesize that the online setting prevents hidden distribution-shift failures, while offline regularization can only guard the limited slice of input space it was measured on. Our method provides robust protection against distribution shift while preserving model performance during supervised finetuning.

## 1 Introduction

Direct Preference Optimization (DPO) has emerged as a simple and effective alternative to reinforcement learning for aligning language models with human preferences Rafailov et al. (2023). By directly optimizing a policy to favor preferred responses over less-preferred ones, DPO avoids the complexity of training a reward model and enables efficient offline alignment. However, DPO's reliance on static, limited preference datasets may introduce a overfitting to the preference data. Since preference data often covers only a narrow slice of the data distribution—and is typically much smaller than the supervised fine-tuning (SFT) corpus—DPO can over-specialize to the preference data and degrade on broader tasks.

While DPO includes a KL divergence term to constrain the policy relative to a reference model, this regularization is applied only on preference examples. As a result, it provides limited protection against distribution shift outside of that subset. To address this, we propose applying KL regularization on non-preference data, which better constrains the learned policy across the full input distribution and improves generalization.

Furthermore, we extend this approach with online KL regularization, where we sample rollouts on non-preference inputs during training and compute KL divergence relative to the reference model. This dynamic regularization helps prevent hidden overfitting and ensures stable optimization throughout training.

In summary, our contributions are twofold:

- We propose KL regularization on non-preference data to improve the generalization and robustness of DPO-trained models.

- We introduce an online KL regularization mechanism that dynamically samples rollouts from non-preference data, enabling continuous control of distributional shift during training.

## 2 Related Works

**Preference-based Learning.** Learning from human preferences aligns AI behavior with human values by training a reward model from human comparisons of model outputs, then optimizing the AI using reinforcement learning. Proximal Policy Optimization (PPO) Schulman et al. (2017) stabilizes this process with conservative policy updates, while Group Relative Policy Optimization (GRPO) Shao et al. (2024) enhances it by learning from ranked group preferences rather than simple pairs.

**Direct Preference Optimization.** Direct Preference Optimization (DPO) introduced by Rafailov et al. (2023) represents a significant advancement in preference learning by directly optimizing policy parameters without requiring explicit reward model training.

**Regularization in Direct Preference Optimization**. However, several limitations of the original DPO formulation have been identified and addressed in subsequent work Azar et al. (2024). Wang et al. (2023) extend DPO beyond its original reverse KL formulation, exploring diverse divergence constraints that can improve training stability and performance. Their work demonstrates that different divergence measures can lead to distinct optimization behaviors and generalization properties. These approaches help prevent overfitting to the preference dataset, ensuring the model doesn't deviate too far from reasonable behaviors during training. However, since DPO relies on static offline preference data, it may struggle with insufficient coverage of the full input distribution.

## 3 Method

We explore two different approaches to dealing with the overfitting of DPO in the presence of extreme data scarcity, and compare them to performing standard DPO in this challenging regime.

The original DPO objective is Rafailov et al. (2023)

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left( \beta \left[ \log \frac{\pi_\theta \left( y^+ \mid x \right)}{\pi_{\text{ref}} \left( y^+ \mid x \right)} - \log \frac{\pi_\theta \left( y^- \mid x \right)}{\pi_{\text{ref}} \left( y^- \mid x \right)} \right] \right) \tag{1}$$

where $\pi_\theta$ is the learned policy, $\pi_{\text{ref}}$ is the fixed reference model, $\sigma$ denotes the sigmoid function, and $\beta$ is the temperature parameter that controls the strength of the preference signal. For our training, we set the hyperparameter $\beta = 0.1$.

As we describe later, training with this objective on the a significantly-ablated dataset of preference data encourages models to overfit, since ordinarily stochastic preferences can be viewed as deterministic.

The first approach we explore to reduce overfitting is to add

We introduce to incorporate online sampling for KL regularization on non preference datasets (i.e. the SFT dataset), which allows the policy to adapt to its own evolving distribution while retaining the efficiency benefits of offline preference learning for the primary objective.

To address overfitting when preference data covers only parts of the original distribution, we use KL regularization using examples from a broader non-preference dataset Wang et al. (2023). The KL-regularized DPO objective becomes:

$$\mathcal{L}_{\text{KL-DPO}} = \mathcal{L}_{\text{DPO}} + \lambda \mathbb{E}_{z \sim \mathcal{D}_{\text{general}}} \left[ \text{KL} \left( \pi_\theta(z) \| \pi_{\text{ref}} \left( z \right) \right) \right] \tag{2}$$

where $z \sim \mathcal{D}_{\text{general}}$ represents examples sampled from the non-preference dataset, $\lambda$ is the KL regularization weight, and $\text{KL}(\cdot \| \cdot)$ denotes the Kullback-Leibler divergence.

Finally, we make the KL regularization online by sampling rollouts during training rather than using fixed offline data. The online KL-regularized DPO objective is:

$$\mathcal{L}_{\text{Online-KL-DPO}} = \mathcal{L}_{\text{DPO}} + \lambda \mathbb{E}_{y \sim \pi_\theta} \left[ \text{KL} \left( \pi_\theta(y) \| \pi_{\text{ref}} \left( y \right) \right) \right] \tag{3}$$

where $y = (y_1, \ldots, y_T)$ is a complete rollout sampled online from $\pi_\theta$ given prefix $y_{<t} = (y_1, \ldots, y_{t-1})$ at decoding step $t$, with prefixes sampled from $\mathcal{D}_{\text{general}}$. This online approach prevents hidden distribution-shift failures by continuously sampling from the evolving policy during training.

# 4 Experimental Setup

We went through these stages as outlined in the default project description.

1. Performing supervised fine-tuning (SFT) on Qwen/Qwen2.5-0.5B base model to adapt to question-answering (QA).
2. Using DPO to align the SFT model to human preferences.
3. Comparing methods for performing DPO with the SFT model as reference model where preference data may not represent the entire SFT data distribution.

## 4.1 Data

In order to fine-tune the Qwen/Qwen2.5-0.5B base model for the question-answering task, we performed supervised finetuning on the Smoltalk dataset Allal et al. (2025). With 460k training examples and 2k validation examples, the SmolTalk dataset provides are broad distribution of responses for instruction following tasks. To perform Direct Preference Optimization (DPO) on our SFT-finetuned model during the reinforcement learning phase, we used the 'train_prefs' and 'test_prefs' HuggingFace's UltraFeedback dataset Cui et al. (2023), which provides 61k training and 2k validation examples across question-answering tasks.

As our extension concerns performing DPO and DPO-adjacent fine-tuning protocols in cases of extreme data scarcity, all models are trained on a substantially ablated version of the UltrafeedBack dataset. For this purpose, we use the first 10% subset (6110 examples) of the training set of UltrafeedBack. In these explorations, we consider ways to supplement the traditional DPO training pipeline with data with instruction-following data that has not been labeled with preferences. In the form of added regularization terms (to be described later), our modifications of DPO also make use of the entire training set of the SmolTalk used in the first part of our project of our exploration.

Testing for the extension part of the project was performed on the same 'test_prefs' slice of the UltraFeedback dataset the rest of the testing work was based upon.

## 4.2 Implementation

We implemented dedicated compute_loss functions for each algorithm (SFT, DPO, RLOO) with unit test verification using pytest. Task-specific tokenization functions handle data formatting and batching, with user tokens masked during loss computation and sequences truncated to 768 tokens. We integrated datasets via Lightning DataModule with distributed PyTorch DataLoaders and implemented corresponding LightningModules Falcon and The PyTorch Lightning team (2019) with custom training/validation steps logging relevant metrics. SFT training used standard hyperparameters (lr=5e-5, cosine schedule, Adam optimizer, 100 epochs with early stopping) on pretrained Qwen0.5B weights. DPO training followed Rafailov et al. (2023) with $\beta$=0.1, KL weight=0.1, lr=1e-6, 150-step warmup, batch size=120, and RMSprop optimizer, training for 5 epochs on UltraFeedback with gradient clipping (val=1.0) for stability. We keep the best checkpoint based on winrate (higher log probability for the preference during validation). Further details can be found in the Appendix. For the online KL regularization we sample from the current parameters with temperature of 1.0, top_p of 0.8 and repetition_penalty of 1.05. We masked out the query tokens in the loss computation in all experiments.

## 4.3 Evaluation Pipeline

### 4.3.1 Model evaluation

We evaluated the model during SFT training with the loss and perplexity. For DPO training we use the DPO loss, the crossed entropy loss and perplexity of the preferred and the unpreferred, preference margin, and the win rate based on logits.

We define the win rate during training as a higher difference of the preferred log probabilities compared to the difference of the dispreferred log probabilities.

Let $\text{log\_pref}_i = (\pi_\theta(x_i^+) - \pi_{\theta_{\text{ref}}}(x_i^+))$ and $\text{log\_unpref}_i = (\pi_\theta(x_i^-) - \pi_{\theta_{\text{ref}}}(x_i^-))$.

The pairwise win rate is computed as:

$$\text{WinRate} = \frac{1}{N}\sum_{i=1}^{N} 1\left[\text{log\_pref}_i > \text{log\_unpref}_i\right]$$

where $1[\cdot]$ is the indicator function, equal to 1 if the condition inside is true, and 0 otherwise.

#### 4.3.2 Leaderboard submission generation

For the generative evaluation we followed the steps in the default project description. This includes using the NVIDIA's 70B-parameter as a judge to evaluation the model answer to the Qwen0.5B-Instruct model. For each prompt, we say that our model wins if the reward score that Nemotron assigned to the response of the model under consideration is greater than that of the reference model. We then summarize the results of this competition with a straightforwardly defined win rate, which the ratio of the number of prompts the model under consideration wins on to the total number of 1000 prompts.

For consistency across different experiments, we use the same set of generation parameters for each response generation for every model. In particular, our decoding process uses a beam search with a temperatuer of 0.6, a TopP of 0.95, and TopK of 20. The prompts vary significantly in length; due to the small context lengths of the models, all prompts are truncated to 512 tokens, and all responses are abrubtly ended after this same number of 512 tokens. We subjectively observe that this truncation length generally leads to models being able to produce complete responses without running out of space. Please see more details in the appendix.

## 5 Results

### 5.1 Quantitative Evaluation

#### 5.1.1 Supervised Finetuning

We measured performance mainly with the cross entropy loss. With SFT training the validation loss was reduced from 1.03 to 0.88. We did not train the SFT model fully due to compute constraints. The learning curves can be found in the appendix 1.

#### 5.1.2 DPO

Our SFT model compared against itself has a win rate of 0.458 and the DPO model trained on the full UltraFeedback dataset achieves a win rate of 0.655. The DPO training significantly improves preference alignment while maintaining comparable perplexity on general datasets, demonstrating effective preference learning without catastrophic forgetting.

#### 5.1.3 Extension

Table 1 shows results for DPO trained on the full UltraFeedback dataset, while Table **??** presents our main experimental results using only 10% of the preference data to simulate extreme data scarcity scenarios.

As shown in Table 2, when training with only 10% of preference data, standard DPO achieves a 0.620 win rate against SFT but shows signs of overfitting with reduced perplexity on preference data. Our offline KL-regularized DPO improves the win rate to 0.650 while better maintaining perplexity across datasets. The online KL-regularized variant achieves a comparable win rate (0.644) with the most consistent perplexity scores across all evaluation datasets, indicating superior generalization. Notably, both KL-regularized methods maintain better alignment with the Qwen0.5B-Instruct model while improving over the SFT baseline, suggesting they preserve beneficial pre-training knowledge while incorporating preference learning.

Table 1: Performance Comparison using win rate against SFT model on 100% of DPO model. Win-rate is calculated based on logits.

| Method | Win-rate | Perplexity on Ultrafeedback | Perplexity on Smolltalk |
|---|---|---|---|
| SFT | 0.500 | 4.41 | 5.06 |
| DPO | 0.655 | 4.48 | 5.06 |

Table 2: Performance Comparison using win rate (defined in section 4.3.1) against SFT model on 10% of the UltraFeedback dataset (6k) with 512 token length.

| Method | KL weight | Win-rate | | Perplexity | | |
|---|---|---|---|---|---|---|
| | | vs SFT | vs Instruct | UltraFeedback preferred | Smolltalk | Wikitext |
| SFT | 0 | 0.500 | 0.493 | 4.41 | 5.06 | 25.31 |
| Qwen0.5B-Instruct | 0 | 0.507 | 0.500 | 3.78 | 5.06 | 25.81 |
| DPO (10% data) | 0 | 0.620 | 0.498 | 4.39 | 4.407 | 25.30 |
| DPO-kl offline (10% data) | 1.0 | 0.650 | 0.470 | 4.40 | 4.86 | 25.27 |
| DPO-kl online (10% 4ata) | 1.0 | 0.644 | 0.467 | 4.41 | 4.85 | 25.24 |

### 5.2 Qualitative Evaluation

Vijay's part. We did not get to do extensive qualitative evaluation due to AWS compute constraints.

## 6 Discussion

While our method improves generalization under data constraints, it requires generating rollouts online, which increases computational cost. Also, KL divergence is a proxy for behavioral alignment and may not always capture semantic differences in generation. Future work could explore adaptive KL schedules.

In future work we would be excited to test our method and validate our results with rollouts and large scale rewards models. For now we only include metrics based in log probabilities on the validation set.

## 7 Conclusion

The online KL regularized objective results in the model being less overconfident (preference margin) with a higher preference win rate. Offline KL regularization interpolates between standard DPO and online KL. Training becomes less stable (though also LR higher after warmup)

## 8 Team Contributions

- **Sophie Ostmeier:** Training setup, including SFT, DPO and RLOO implementation, training of all models, evaluating all models (loss, perplexity, etc. ) on different datasets, extension idea, implementation and training of the models, evaluation of the extension, writing of the paper except the evaluation pipeline section.
- **Vijay N. Joseph:** Generation of prompts for the leader board submissions

**Changes from Proposal** Initially we planned to use the default project setup for a costum project for efficient decoding strategies. However, dur to some difficulty with the evaluation setup, we decided to use the default project setup to focus on some technical innovation in the extension.

## References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav,

et al. 2025. SmolLM2: When Smol Goes Big–Data-Centric Training of a Small Language Model. *arXiv preprint arXiv:2502.02737* (2025).

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4447–4455.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, et al. 2023. Ultrafeedback: Boosting language models with scaled ai feedback. *arXiv preprint arXiv:2310.01377* (2023).

William Falcon and The PyTorch Lightning team. 2019. *PyTorch Lightning*. `https://github.com/Lightning-AI/pytorch-lightning`

Raphael Rafailov, Alex Go, Faisal Ladhak, Yoonho Kim, Deep Ganguli, and Tatsunori B. Hashimoto. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *arXiv preprint arXiv:2305.18290* (2023).

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. In *arXiv preprint arXiv:1707.06347*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

Chaoqi Wang, Yibo Jiang, Chenghao Yang, Han Liu, and Yuxin Chen. 2023. Beyond reverse kl: Generalizing direct preference optimization with diverse divergence constraints. *arXiv preprint arXiv:2309.16240* (2023).

## A SFT learning curve

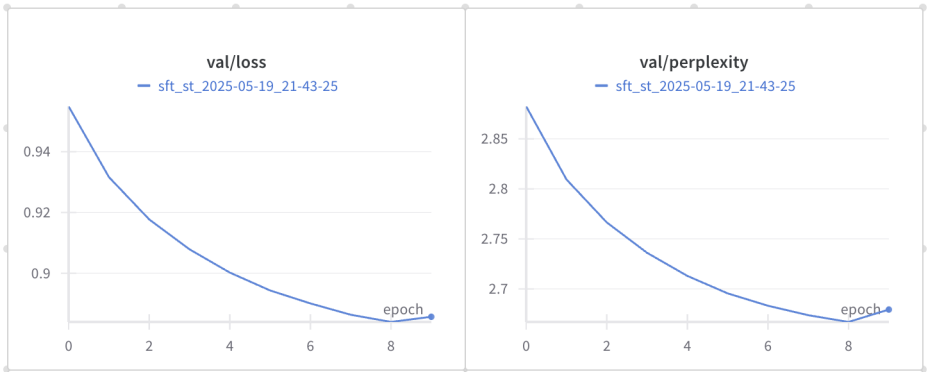


Figure 1: Training of SFT model, not fully converged. However loss and perplpexity are low in absolute terms.

## B Implementation and Evaluation Details

For each algorithm, we implemented a dedicated `compute_loss` function and verified its correctness using unit tests with dummy data. These tests were organized into separate files (`test_sft.py`, `test_dpo.py`, `test_rloo.py`, and `test_reward_rloo.py`) and executed using `pytest -s` to ensure correct functionality and expected behavior. Here after we decided to focus on DPO.

To prepare the datasets, we developed task-specific tokenization functions for each task and algorithm: `tokenize_fn_sft_cd`, `tokenize_fn_sft_st`, `tokenize_fn_dpo`, `tokenize_fn_dpo_st_kl`, `tokenize_fn _dpo_st_kl_online`. These functions handle formatting and batching of the data.

We kept the tokenizer flag `add_generation_prompt` on during training. We always mask out the user tokens for the loss computation and truncate after 512 tokense due to compute limitations. We integrated them into a Lightning DataModule, where datasets from Hugging Face are loaded into a distributed PyTorch `DataLoader` and subsequently passed to the Lightning CLI for training and validation.

For each algorithm, we implemented a corresponding `LightningModule` that leverages the appropriate `compute_loss` function. We customized the `training_step` and `validation_step` methods to log relevant metrics such as loss and perplexity during both training and evaluation. For DPO, we added metrics like preference margin, preference and rejection loss and win rate.

SFT training was performed using standard hyperparameters: a learning rate of `5e-5` with a cosine learning rate schedule, the Adam optimizer, and a maximum of 100 epochs with early stopping based on validation performance. We used the pretrained weights for the Qwen0.5B available for the huggingface implementation. We use wandb to track training metrics.

For DPO training we trained for 5 epochs keeping the best checkpoint based on the win rate. Inspired by the original DPO we used a beta of 0.1, a 0.1 kl weight, learning rate of 1e-6 with 150 steps warmup, effective batch size of 120 and RMSprop optimizer Rafailov et al. (2023). We first trained on the full Ultrafeedback dataset. For the extension we reduced the dataset to 10%. We used gradient_clip_val=1.0 to stabilize training a bit.

To expand on the steps of the evaluation, we will explain in detail the steps. For a given preference-optimized model, we first generate a response from each of the 1000 prompts in the test slice of the UltraFeedback dataset. Efficient generation to this end is accomplished by using the inference server vLLM, which is able to gain more than an order of magnitude speedup using techniques such as PagedAttention. This efficient generation yields, in under 3 minutes, 1000 prompt/response pairs. To reduce the quality of each prompt/response pair with a single scalar reward point, we query NVIDIA's Nemotron model (hosted on their servers) with each prompt/response pair, and receive a reward score. This reward score has no absolute meaning, so we must have a standard to compare it against to get a robust result. We perform the same set of steps for a reference model, which in this paper is the SFT-finetuned model that our preference-optimized models are built upon.

## C Minimal Data Setting

In the setting of minimal data i.e. just one batch of 120 examples, we observe that the online KL stabilizes training, though the data is not enough to improve validation performance meaningfully (Figure 2).

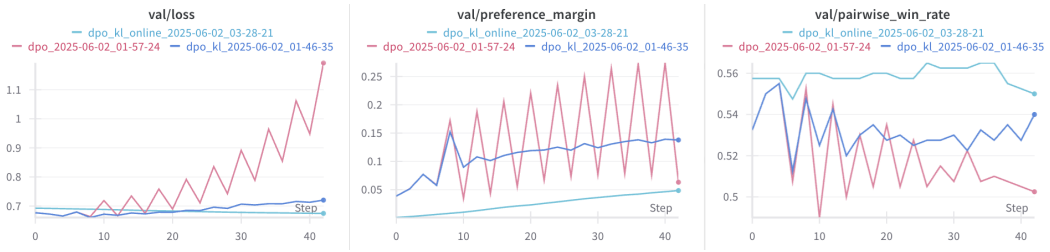


Figure 2: DPO, DPO-kl offline and DPO-kl online with just on batch of data (120 examples)